Ders 9 Genişletilmiş Notu: Veri Tasarımı: Veri Modelleri, Varlık-İlişki Modeli ve Tablolara Dönüştürme

Ders 9 Genişletilmiş Notu: Veri Tasarımı: Veri Modelleri, Varlık-İlişki Modeli ve Tablolara Dönüştürme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Tasarımın Son Konusu ve Sürecin Kalanı
- 2. Neden Veri Tabanı Tasarımı?
- 3. Veri Tabanı Tasarım Süreci
- 4. Veri Modelleri
- 5. Varlık-İlişki Modelinin Sembolleri
- 6. Varlık, Varlık Kümesi ve Tablo
- 7. Öz Nitelik ve Anahtar Öz Nitelik
- 8. İlişkiler
- 9. Personel-Departman Örneği
- 10. İlişki Türleri
- 11. Çoka Çok İlişki Nasıl Gerçeklenir?
- 12. Kitapçı Veri Tabanı Örneği
- 13. Varlık-İlişki Modelini Tablolara Dönüştürme
- 14. Çoka Çok İlişki: Film ve Sinema Örneği
- 15. Zayıf Varlık
- 16. Veri Tasarımının Özeti
- 17. Fizibilite ve Analiz Çalışmasını Belgelemek

1. Tasarımın Son Konusu ve Sürecin Kalanı

Hoca derse, tasarım aşamasında söylenecek son bir söz kaldığını belirterek başladı: Veri tasarımı nasıl yapılır, veri modellemesi nasıl yapılır, kullandığımız veri yapıları, veri tabanları ve veri depoları nasıl tasarlanır? Bu konu çok ince detaylarına girilmeden, hızlıca işlendi. Veri tasarımı da tamamlanınca tasarım başlığı bitmiş olur.

Tasarım bittikten sonra elimizde tasarlanmış bir ürün vardır ve artık bu ürünü üretmemiz gerekir. Hoca sürecin geri kalanını şöyle özetledi:

- **Gerçekleştirme:** Kodlama dersleri tekrar edilmeyecek. Ama kodlamada nelere dikkat edilmesi gerektiği, nereden başlanacağı ve kod kalitesi için birkaç şey söylenecek.
- **Test:** Yapılanların doğruluğuna bakmak, onları sınamak gerekir. Kabaca hangi testlerin neden yapıldığı ve her testin sürecin neresini karşıladığı incelenecek.
- **Kurulum ve geçiş:** Bir bilgi sistemi nasıl kurulur, yani nasıl kullanılmaya başlanır? Mevcutta bir sistem varsa ondan yeni sisteme geçiş nasıl sağlanır?
- **Bakım:** Sistem devreye alındıktan sonra, devrede kaldığı ömür boyunca kullanılır ve çalıştırılır. Ne gibi bakımlar olduğu, adları, ne işe yaradıkları, neden yapıldıkları ve hangi bakımda neyin değiştiği işlenecek.

Test aşamasında bir sorun çıkarsa sürecin neresine dönüleceği sorunun kaynağına bağlıdır. Hoca üç durum saydı:

- Kodda hata bulunduysa düzeltilir ve tekrar test edilir.
- Tasarım yanlış olduysa iki adım geri gidilir. Tasarım güncellenir, tekrar gerçekleştirme yapılır, sonra tekrar test edilir.
- Problem yanlış anlaşıldıysa ve yanlış modellendiyse analize kadar geri gidilir. Yeni bir analiz modeli, yeni bir tasarım ve yeni bir gerçekleştirme gerekir.

  Hocanın vurgusu: Süreçte ne kadar erken safhada bir hata yaptıysak ve bunu ne kadar geç fark ettiysek maliyeti o kadar yüksek olur.

Testlerden geçtikten sonra elimizde işlevsel ve işlevsel olmayan gereksinimleri karşılayan bir tasarımla üretilmiş bir bilgi sistemi vardır. Fakat iş burada da bitmez. Hoca, devreye alınan sisteme bir canlı organizma gibi sürekli bakım yapmak zorunda kalacağımızı söyledi. Bakım, yazılımın kullanıldığı bütün ömür boyunca sürer. Hem en maliyetli aşamadır hem de bize en çok parayı kazandıran aşamadır. Süreçteki son adım budur.

Böylece bütün yol görülmüş olur: Ortada yalnız bir fikir ya da problem varken önce "yapabilir miyiz?" diye bakılır. Yapılabiliyorsa bunun neye mal olacağı incelenir. Sonra problem anlaşılır, çözüm önerisi geliştirilir, gerçekleştirme ve testler yapılır, sistem devreye alınır ve bakımla devrede tutulur.

2. Neden Veri Tabanı Tasarımı?

Hoca "veri tasarımı" yerine kabaca "veri tabanı tasarımı" demesini şöyle açıkladı: Bilgi sistemlerinde verilerin bir yerde durması gerekir. Bunun için veri tabanı yönetim sistemi dediğimiz, verileri tutan özel sistem yazılımları kullanılır. Bu yapılar farklı karmaşıklıkta olabilir:

- Çok basit dosyalar.
- C'deki `struct`'ları dosyaya kaydettiğiniz kayıt yöneticileri.
- Karmaşık ilişkileri bir arada barındıran; tutarlılığı kontrol edip garanti eden; verinin depolanmasını ve erişimini optimize eden sistemler.

Günümüzde bir de NoSQL veri tabanları duyuyoruz. Hoca bunu "SQL, yani bir sorgu dili olmadan nasıl veri tabanı yapılabiliyor?" sorusuyla andı ve özellikle büyük veriyle birlikte hayatımıza giren yenilikçi teknolojiler olarak tanıttı.

Bu derste odak, bilgi sisteminde tuttuğumuz, aldığımız ve ürettiğimiz verilerin durduğu veri tabanları ve özellikle ilişkisel veri tabanı modelidir. Yani burada veri tasarımı denince ilk odak noktası veri tabanı tasarımıdır.

3. Veri Tabanı Tasarım Süreci

Veri tabanı tasarımı ayrı bir iş olarak sonradan yapılmaz. Bütün bilgi sistemi geliştirme süreciyle paralel yürütülür. Hoca adımları slayttaki kutular üzerinden sırayla anlattı.

3.1 Gereksinim Analizi ve Fizibilite

Gereksinim analizinde bile hangi verilerin tutulması gerektiği ve buna yönelik ihtiyaçlar ortaya konmaya başlar. Hatta bunun öncesinde, fizibilite aşamasında, ne gibi yazılım ve donanımlara ihtiyaç duyulacağı ve bunların maliyetleri yavaş yavaş belirlenir.

3.2 Kavramsal ve Mantıksal Tasarım

Ön tasarım ve ayrıntılı tasarım aşamalarına burada kavramsal ve mantıksal tasarım denir. Artık bir çözüm önerisi ortaya konduğu için şu sorular cevaplanır: Bu bilgi sisteminde hangi veriler tutulacak, bunların arasındaki ilişkiler neler olacak? Yani varlık-ilişki veri modelini ortaya çıkarmak için gereken çalışma bu aşamada yapılır. Ayrıntılı tasarıma geçildiğinde de mantıksal model, daha detaylı çözümlemesiyle ve bütün taraflarıyla birlikte oluşturulur.

Hoca, bu dersin ve ders notlarının kabaca bu alanda kalacağını söyledi. Ancak gerçek hayatta bir bilgi sistemindeki verileri tasarlamak için burada kalmak yeterli değildir.

3.3 Normalizasyon

Mantıksal model hazırlandıktan sonra üzerine normalizasyon işlemi uygulanır. Bunun sebepleri şunlardır:

- Veri tekrarını önlemek.
- Veri tutarlılığını sağlamak.
- Performans kriterlerini karşılamak.

Bunun için normal formlar vardır. En çok kullanılan seviyeler birinci normal form, ikinci normal form, üçüncü normal form ve Boyce-Codd normal formudur. Hoca bu ders kapsamında ayrıntıya girmedi, çünkü bunlar veri tabanı derslerinde öğrenilecek. Normalizasyon da yapılınca elimizde mantıksal olarak hazır, performans ve veri güvenliği kriterlerini karşılayan bir veri çözümü olur.

3.4 Veri Tabanı Uyarlaması

Sonra bu çözümü fiziksel hâle dönüştürmek, yani gerçekleştirmek gerekir. Hoca, yazılım dünyasında her şeyin aslında sanal olduğunu ekleyerek "fiziksel" sözünü şu işlerle açıkladı:

- Veri tabanı yönetim sisteminin kurulması ve ayarlarının yapılması.
- Sistem üzerinde sizin veri tabanınızın oluşturulması.
- Varlık kümeleri olan tabloların yaratılması.
- Normalizasyondan ve mantıksal modeldeki tanımlardan gelen kısıtların tanımlanması.
- İlk verilerin yüklenmesi.

Bunlar yapıldığında veri tarafındaki gerçekleştirme de yapılmış olur.

3.5 Veri Tabanı Programlama

Kullanım senaryolarındaki bazı işlevler ve sorumluluklar veri katmanında, verinin üzerinde de çözülebilir. Veri tabanı yönetim sisteminin sağladığı ortamda ufak fonksiyonlar, kodlar yazılır. Bunlar doğrudan verinin üzerinde çalıştığı için sonucu daha hızlı üretir.

Bu, eskiden performans için çok kullanılan bir teknikti. Hoca artık pek sevilmediğini iki gerekçeyle anlattı:

- **Kavramların ayrı tutulması ilkesi:** Çalışan kodun veri katmanında kalması pek istenen bir şey değildir.
- **Büyük veri:** Veriler o kadar hacimli hâle geldi ki, bu tip programlamayı ilişkisel veri tabanı modelleri üzerinde çalıştırmak anlamsız kaldı.

Yine de gerekirse veri tabanı katmanında fonksiyonlar, işlevler ve prosedürler yazılarak çalışan kod gerçekleştirilebilir.

Bütün adımlar birlikte düşünüldüğünde veri tabanı tasarımı şu sorularla ilerler: Bize ne lazım ve bu bize neye mal olur? Neleri tutmam ve bunları nasıl ilişkilerle bağlamam gerekir? Üzerinde nasıl iyileştirmeler yapmalıyım? Bunlar nasıl gerçekleştirilip hizmet sunabilir hâle getirilir?

4. Veri Modelleri

İlişkisel bir veri tabanı oluşturmak, yani çeşitli verileri varlık olarak modelleyip haklarında veri tutmak ve onları birbiriyle ilişkili birimler hâline getirmek için bir veri modeli kullanmak gerekir. En yaygın karşımıza çıkan model, varlık-ilişki modelidir (Entity Relationship Model). Veri tabanında fiziksel olarak tutulacak verilerin tasarımının nasıl yapılacağı bu modelle söylenir. Başka modeller de vardır ve hoca bunları kısaca tanıttı.

4.1 Sıra Düzensel Model

Eskiden dosya sistemleri üzerinde kayıt kütükleri, yani record manager'lar vardı. Hoca bunlara sıra düzensel veri modeli dedi. C'deki `struct`'ları düşünün: Bu yapıların dosya sistemi üzerinde tutulmasını, yaratılmasını, silinmesini, erişilmesini ve sorgulanmasını sağlayan, dosya üzerindeki kayıtlardan oluşan bir modeldir. Günümüzde çok fazla kullanılmıyor.

> **Ders sonrası not:** Hoca sıra düzensel modeli dosyalardaki kayıt yöneticileri ve `struct`'larla açıkladı. "Sıra düzensel" büyük olasılıkla hiyerarşik modelin Türkçe karşılığıdır. Hiyerarşik modelin asıl özelliği de büyük olasılıkla kayıtların ağaç yapısında, ebeveyn-çocuk ilişkisiyle düzenlenmesidir. Dosyada düz kayıt tutmak tek başına bunu sağlamaz.

4.2 XML ve JSON: Yarı Yapılı Veri

XML (Extensible Markup Language, genişletilebilir işaretleme dili) yarı yapılı bir veri modelidir. Yarı yapılıdan kasıt, verinin ve veriyi tanımlayan üst verinin aynı anda bir arada bulunmasıdır. Veri depolamak için kullanılabilir, ama çok verimli olmaz. Daha çok farklı platformlar ve farklı bilgi sistemleri arasında veri alışverişi için kullanılır.

XML bundan 15-20 yıl önce inanılmaz popülerdi ve endüstri standardı gibiydi. Hâlâ çok kullanım alanı var. En basitinden Android'deki manifest dosyalarına bakarsanız onların da XML yapısında olduğunu görürsünüz.

Fakat XML'in overhead denen yükü fazladır. Bir veriyi tutmak, tanımlamak ve taşımak için o veriyi tanıtan her şeyi, nasıl geçerleneceğini anlatan ilave dosyaları ve gösterim için stil tanımlarını da oluşturmak gerekir. Bunlar işin içine girince ufak bir veri için bile kocaman bir trafik ve depolama ihtiyacı doğar.

> **Ek bilgi (derste anlatılmadı):** Hocanın andığı geçerleme dosyaları büyük olasılıkla DTD veya XSD şema dosyalarıdır. Stil tanımları da büyük olasılıkla XSL veya CSS dosyalarıdır.

Bu modelin daha hafif hâlleri de vardır. Bunlardan biri JSON'dur (JavaScript Object Notation). XML gibi yarı yapılıdır, ama çok daha hafiftir. Günümüzde heterojen sistemler arasında veri aktarımı için pek çok uygulamada neredeyse standart olarak kullanılıyor.

4.3 Nesne Modeli ve Object Relational Mapping

Nesneye yönelik programlamanın gelişmesiyle birlikte nesne modeli bir ara popüler olmaya çalıştı. Object database denilen bu çözümlerde, uygulamadaki nesnelerin örnekleri (instance) olduğu gibi bir nesne veri tabanında depolanıyordu. Hocaya göre bu çok tutmadı, iki sebeple:

- Dünya yalnız nesneye yönelik değildir. Gömülü sistemler gibi işlevsel çözümlemenin sürdüğü pek çok alan vardır.
- İlişkisel veri tabanları da boş durmadı. Hem çok güçlüler hem de güçleri sürekli artıyor.

Yine de bu fikir, arada hibrit bir çözümün doğmasına yol açtı: Object Relational Mapping. Uygulama nesneye yönelik geliştirilir, arkada ise varlık-ilişki modeline dayanan klasik bir ilişkisel veri tabanı kullanılır. İkisi arasındaki dönüşümü bir ara katman yapar. En bilinenlerinden biri Hibernate'tir. Veri tabanı tablolarındaki yapıyı sizin nesne olarak görüp erişmenizi ve kullanmanızı sağlar.

4.4 NoSQL ve Bellek İçi Çalışan Sistemler

Günümüzde NoSQL veri tabanları, MongoDB, AllegroGraph ve Redis gibi in-memory çalışan pek çok yapı kullanıma girdi. Hoca bunları varlık-ilişki modelinin hiçbir zaman alternatifi olarak değil, destekçisi olarak tanıttı.

Bunların neden gerektiğini hoca bir örnekle anlattı. Facebook, Twitter ya da Zoom gibi bir uygulama veya çevrim içi bir eğitim sistemi tasarladığınızı düşünün. Burada, örneğin video verileri nedeniyle, hacim olarak inanılmaz büyük bir veri vardır. Bunu ilişkisel veri tabanında tutarsanız çok büyük yatırım yapmak zorunda kalırsınız. Örneğin kapasite dolayısıyla Oracle'a çok yüksek lisans bedeli ödemeniz gerekebilir. MongoDB gibi NoSQL veri tabanları veya AllegroGraph gibi yapılar bunu engellemek için, özellikle büyük veri mimarilerine uygun olarak önerilir.

Öte yandan bilgi sistemlerinin içinde mutlaka bir parça ilişkisel veri tabanı durmaya devam eder. Çünkü business logic dediğimiz, gereksinimleri yerine getirmek için gereken verileri ilişkisel tablolarda tutmak en doğru yöntemdir.

Hocanın vurgusu: Bir bilgi sisteminde "yalnız NoSQL kullanacağım" ya da "yalnız MongoDB kullanacağım" demek zorunda değilsiniz. Hacimli veya zamanla değişen veriler için bu yeni çözümleri, normal iş katmanının çalışması için de klasik bir veri tabanı yönetim sistemini kullanabilirsiniz. Günümüzde tasarımlar zaten bu şekilde yapılıyor.

5. Varlık-İlişki Modelinin Sembolleri

Varlık-ilişki modelinin sembolleri çok basittir. Dört sembol vardır ve bunlar çizgilerle birbirine bağlanır:

Kavram	Sembol	Açıklama
Varlık (varlık sınıfı)	Dikdörtgen	Bilgi sisteminde hakkında veri tutulan şey
Öz nitelik	Oval	Varlığın taşıdığı özellik; varlığın etrafına papatya gibi serpiştirilip bağlanır
Anahtar öz nitelik	Altı çizili ad taşıyan oval	Varlığı benzersiz olarak tanımlar
İlişki (bağıntı)	Eşkenar dörtgen (baklava)	Varlıkları birbiriyle ilişkili hâle getirir

Öz niteliklerden biri bizim için özellikle değerlidir, çünkü varlığı benzersiz olarak tanımlamamızı, ona erişmemizi ve diğerlerinden ayırt etmemizi sağlar. Buna primary key, yani birinci anahtar denir. Hoca, önemli şeylerin altı çizildiği için bunun da adının altının çizildiğini söyledi.

6. Varlık, Varlık Kümesi ve Tablo

Varlık, bu modelin en temel üyesidir. Bilgi sistemimizde var olan ve benzerlerinden ayırt ettiğimiz her şey bizim için bir varlıktır. Örneğin bir kitap, bir öğrenci, bir araba. Yani gerçek dünyada olduğu gibi.

Hoca burada nesneye yönelik analizle bağlantı kurdu. Orada gerçek problemin dünyasındaki kavramları mümkün olduğunca birebir yazılım nesnelerine dönüştürmeye çalışıyorduk. Hakkında bilgi tuttuğumuz ya da üzerinde işlem yaptığımız kavramsal sınıflar, veri depolarken de varlık olarak modellenebilir. Değişikliğe uğrayabilirler, ama genel olarak oradaki iş nesneleri birer varlık adayıdır.

Fiziksel olarak veri tabanı üzerinde düşündüğümüzde, verileri tutan temel yapı olan tablo, varlıkların kümelenmesiyle oluşur. Yani bir veri tabanı tablosu, varlık-ilişki modelindeki bir varlık kümesine eşittir. Bütün kitap varlıkları bir tabloda tutulur ve kitap varlık kümesi kitap tablosunu oluşturur. Öğrenci tablosu ve araba tablosu da böyledir.

7. Öz Nitelik ve Anahtar Öz Nitelik

Varlıklar tek başına yeterli değildir. Onlar hakkında çeşitli veriler, yani onlarla ilişkili özellikler tutulmalıdır. Varlığın her bir özelliğine nitelik ya da öz nitelik denir. Örneğin bir öğrenci varlığının adı, sınıfı, numarası ve ağırlıklı genel not ortalaması vardır. Bölümü de olabilir, çift ana dal gibi başka bir şeye dahil de olabilir. Hakkında bilmeniz gereken bütün özellikler öz nitelikleri oluşturur. Ovallerle gösterilir, içine adı yazılır. Veri tabanı açısından düşünüldüğünde, tablodaki her bir sütun bir öz niteliği gösterir.

Öz niteliklerden öyle biri tanımlanmalıdır ki aldığı değer, kümedeki her bir varlık için benzersiz olsun. Yani tek olsun, biricik olsun. Bu yüzden adının altı çizilerek gösterilir.

Hocanın örneği, dört niteliği olan bir personel varlık kümesidir: sicil numarası, ad, görev ve maaş. Personellerin adı aynı olabilir, görevi aynı olabilir, maaşı aynı olabilir. Ama her personele benzersiz bir sicil numarası atanırsa bu numara anahtar nitelik olur. Kümenin içinden tek

bir özel varlığa erişmek gerektiğinde, örneğin onun adını öğrenmek için, veri tabanı yönetim sistemine soru anahtar nitelik kullanılarak sorulur.

8. İlişkiler

Elimizde birbirinden bağımsız bir sürü varlık kümesi vardır. Öğrencinin bölümü bir nitelik olarak düşünülebilir. Çift ana dal yapan öğrencinin iki bölümü bile olabilir. Öğrenciler ders de alır. O zaman ders de öğrencinin bir niteliği olmalı gibi görünür.

Fakat derse bakınca onun da ayrı bir kavram olduğu görülür. Dersin kodu, adı, kredisi, AKTS'si, sınav sayısı, dönemi, hangi sınıfta açıldığı ve hocası vardır.

Kendine göre farklı öz nitelikleri olduğuna göre ders de ayrı bir varlık kümesidir. Öte yandan öğrencilerin ders aldığını da biliyoruz, yani öğrenci varlığı ile ders varlığı arasında bir ilişki vardır.

İşte varlıkların bu şekilde bir araya gelmesine bağıntı ya da ilişki denir. Modelin adı da zaten bu ikisinden, varlık ve ilişkiden oluşur. İlişki bir eşkenar dörtgenle gösterilir ve içine ilişkinin adı yazılır. Genellikle 1'e N, 1'e 1 ya da N'e N gibi çoğullamalarla görülür. Bunlar varlık kümelerinin gerçekleştirmeleri olan tablolar arasındaki ilişki tipleridir.

9. Personel-Departman Örneği

İlk varlık-ilişki örneğinde iki varlık kümesi vardır: personel ve departman, yani bölüm.

Personel: Sicil numarası benzersiz tanımlayıcı, yani birinci anahtardır. Ayrıca isim ve kazanç tutulur. Hoca modeli okurken şuna dikkat çekti: Bir öz niteliğin modelde olması, sistemde onunla bir iş yapıldığını gösterir. İsim tutuluyorsa isimle ilgili bir iş vardır. Kazanç tutuluyorsa sorgulanması, yazdırılması ya da onu soran başka sistemlere bildirilmesi gerekiyordur. Personelin kaç yıllık olduğu, boyu ve kilosu modelde yoktur, çünkü bu sistemin o verileri kullanarak işlem yapmasına gerek yoktur.

Departman: Numarası (örneğin bizim bölüm için 011), adı (örneğin Bilgisayar Mühendisliği) ve bütçesi tutulur. Hoca bütçeye örnek olarak, ufak tefek harcamalar için dekanlık üzerinden bölümlere yılda bir kez aktarılan küçük bütçeyi gösterdi.

Çoğullama: Bir personel bir ya da birden fazla departmanda çalışıyor olabilir. Sistemin yapısına göre yalnız tek bir departmanda çalışmak zorunda da olabilir. Buna tasarımı yapan biz karar veririz. Bir departmanda ise büyük ihtimalle birden fazla personel olacaktır. Hiç çalışanı olmayan bir departman da olabilir, bu durumda sayı sıfır ile N arasındadır.

Personel departmanda çalışır, departman da personel barındırır. O hâlde ikisinin arasında eşkenar dörtgenle gösterilen bir "çalışır" ilişkisi tanımlanır. Artık "şu sicil numaralı personel şu departmanda çalışıyor" ya da "şu kodlu departmanda çalışan personeller şunlar" diyebiliriz.

9.1 İlişkinin Kendi Öz Nitelikleri

İlişkinin de varlık kümeleri gibi kendine ait öz nitelikleri olabilir. Hoca kendi adıyla örnek verdi: Göksel, Bilgisayar Mühendisliği departmanında 2003'ten beri çalışıyor. Başka bir departmana geçtiğini varsayalım: aynı sicil numaralı Göksel, Biyomedikal Mühendisliği bölümünde 2024'ten beri çalışıyor olsun. Başlangıç tarihi çalışma ilişkisinin kendine has bir niteliğidir. Bu nedenle hiçbir varlığa değil, doğrudan ilişkinin kendisine tanımlanabilir.

> **Ders sonrası not:** Başka bölüme geçiş varsayımsal bir örnektir. Kayıt 2022-2023 bahar döneminden olduğu için 2024 yılı da gerçek bir tarih değil, örneğin parçasıdır.

Hoca bu modelin esnek ve gösteriminin çok basit olduğunu söyledi. Neler hakkında veri tutulacağı, hangi verilerin benzersiz olacağı ve varlıkların nasıl bağlanacağı çizilerek gösterilir.

10. İlişki Türleri

10.1 Bire Bir (1:1)

Bir varlık kümesindeki her bir varlık, diğer varlık kümesinde en çok bir varlıkla ilişkili olabilir. Birden fazla varlıkla ilişki içinde olamaz.

Hocanın örneği evlilik ilişkisidir: Bir tarafta erkeklerden oluşan, öbür tarafta kadınlardan oluşan bir varlık kümesi vardır. Küme diyagramında bir erkekten bir kadına tek çizgi çizilir, aynı erkekten iki çizgi çizilemez. Hiç ilişkisi olmayan bir varlık da olabilir, ama en fazla bir ilişkisi olur. Uygulamalarda bire bir ilişki pek karşımıza çıkmaz. Daha çok bire çok ve çoka çok ilişkiler görülür.

10.2 Bire Çok (1:N)

Hoca küme diyagramını şöyle okudu: Sağdaki kümede tek bir elemana bakın. Karşı taraftan bu elemana en fazla bir çizgi gelir. Ama soldaki bir eleman, sağdaki iki ayrı elemanla ayrı ayrı ilişkide olabilir, ondan iki çizgi çıkabilir. Hiç ilişkisi olmayan elemanlar da olabilir. İlişki iki yönde de okunabilir: 1'e N ya da M'e 1.

> **Ders sonrası not:** Hoca tanımı "bire çok ilişkide bir kümedeki her bir varlık diğer kümede en çok tek bir varlıkla ilintili olabilir" diye verdi. Bu cümle yalnız N tarafını anlatır. Doğrusu şöyledir: N tarafındaki her varlık, 1 tarafında en çok bir varlıkla ilişkilidir. 1 tarafındaki bir varlık ise N tarafında birçok varlıkla ilişkili olabilir. Hocanın küme diyagramı anlatımı da böyledir.

Örnek öğrenci ve bölüm ilişkisidir. Çift ana dal gibi bir durum olmadığını, bir öğrencinin yalnız tek bölümde okuyabildiğini varsayın. Hoca bunu lisedeki sayısalcı, sözelci, eşit ağırlıkçı ayrımına benzetti. Bir bölümde N tane öğrenci okuyabilir, ama bir öğrenciden yola çıkarsanız yalnız tek bir bölüme varırsınız. Pek çok öğrenci Bilgisayar Mühendisliği'nde okur, başka öğrenciler başka bölümde okur. Bir öğrenci buradan çıkıp iki ayrı bölüme gidemez.

10.3 Çoka Çok (N:M)

Varlık kümelerinden her biri, diğer kümedeki birçok varlıkla ilişkili olabilir. Buna N'e N ya da N'e M denir, ikisi de doğrudur.

Örnek öğrenciler ve derslerdir. 2042 kodlu bu dersi 80-90 öğrenci alıyor. 2512 kodlu Veri Yapıları dersini başka öğrenciler alıyor, ama bazı öğrenciler aynı olabilir. Bir öğrenci hem Veri Yapıları'nı hem Sistem Analizi'ni alıyor olabilir. Özetle bir öğrenci N tane ders alır, bir derste N tane öğrenci olabilir. Her iki tarafta da birden fazla varlıkla ilişki vardır.

11. Çoka Çok İlişki Nasıl Gerçeklenir?

Hoca, ayrıntısını birkaç slayt sonra göstereceği konuyu önce özetledi. Varlık kümelerinin veri tabanındaki karşılığı tablolardır. Tablodaki her bir satır bir varlığa, her bir sütun bir öz niteliğe karşılık gelir.

Çoka çok ilişkinin her bir tekrarını benzersiz olarak ayırt edebilmek için iki varlık tablosu yetmez. İki varlık artı bir de çoklu ilişki vardır. Bu yüzden eşkenar dörtgenle gösterilen ilişki için de ayrı bir veri tabanı tablosu oluşturulur. Bu tablo varlıklara benzersiz anahtarlar üzerinden bağlanır. İki benzersiz anahtar bir araya getirildiğinde ilişki tablosu içinde benzersiz bir tanımlayıcı elde edilebilir.

12. Kitapçı Veri Tabanı Örneği

Bir kitap satış uygulaması yapmak istediğinizi düşünün. Depolanması gereken varlıklar kitaplar, yazarlar ve yayın evleridir.

Varlık	Öz nitelikler	Anahtar
Kitap	ISBN, başlık, yayın evi ID, fiyat	ISBN

Varlık	Öz nitelikler	Anahtar
Yazar	yazar ID, ad, soyad, telefon	yazar ID
Yayın evi	yayın evi ID, ad, telefon	yayın evi ID

İlişkiler şöyledir:

- **Kitap-yazar, çoka çok (“tarafından yazılmış”):** Bir yazar N tane kitap yazabilir, bir kitap birden çok yazar tarafından yazılmış olabilir.
- **Kitap-yayın evi, bire çok:** Bir yayın evi N tane kitap yayınlayabilir. Kitaptan yayın evine doğru okununca da bir kitap yalnız bir yayın evine aittir.

Hoca, kitaptaki yayın evi ID alanını diyagrama bilerek fazladan koyduğunu söyledi. İlişki diyagramda zaten gösterildiği için bu alanı göstermek zorunlu değildir.

Soru: Hoca kendisi öğrenci ağzından sordu: Dostoyevski’yi 50 yayın evi basmıyor mu? O zaman bir kitap nasıl tek yayın evine ait olur?

Cevap: Basıyor, fakat bu baskıların ISBN numaraları farklıdır. Özdeş görünseler bile rengi, kağıdı, kalınlığı ve fontu başkadır. Bu yüzden böyle bir bilgi sisteminde bağımsız kitap varlıkları sayılırlar.

Hoca nesneye yönelik tasarımla bir bağlantı daha kurdu. Orada ürünün özellikleri kaybolmasın diye ayrı bir “ürün tanım” sınıfı yapılmıştı. Burada da bir kitap tanımı, başlığı ya da şablonu ile gerçek fiziksel kitaplar arasında böyle bir ayrıma gidilebilir. Bu diyagramda bu ayrım yapılmamıştır.

12.1 Bire Çok İlişkide Yabancı Anahtar

Bir kitap yalnız bir yayın evinde olabileceğine göre, kitabın yayın evi bir şekilde takip edilebilmelidir. Bunun için yabancı anahtar denen bir ID kullanılır. Bu ID, bağlı olunan taraftaki tekil varlığın kod numarasıdır. “ISBN numarası şu olan kitabın yayın evinin adı nedir?” sorusu şöyle cevaplanır:

1. ISBN benzersiz olduğu için varlık kümesinin içinde o tekil kitaba erişilir.
2. Kitabın tablodaki yayın evi ID kolonu okunur. Hocanın örneğinde bu değer 345’tir.
3. Yayın evi tablosunda ID’si 345 olan satıra gidilir, adı veya telefonu okunur.

Kitap ile yayın evi arasındaki bire çok ilişki bu sayede tanımlanmış olur.

12.2 Çoka Çok İlişkide Ayrı Tablo

Kitap ile yazar arasındaki çoka çok ilişki için ayrı bir tablo, yani ayrı bir varlık kümesi oluşturulur. Bu tablodaki her satır “ISBN numarası şu olan, yazar ID’si şu olan bir kayıt” anlamına gelir.

- Aynı ISBN numarası için ikinci ve üçüncü yazarın satırları olabilir. İstenirse tabloya yazar sırası gibi bir sıra numarası da konabilir.
- Benzer şekilde bir yazarın N tane kitabı olabilir. Hocanın örneğinde 12 ID’li yazar için şu ISBN’li kitap, başka bir ISBN’li kitap ve yine başka bir ISBN’li kitap diye üç satır vardır.

13. Varlık-İlişki Modelini Tablolara Dönüştürme

Kavramsal veri tasarımı bir varlık-ilişki diyagramı olarak yapıldıktan sonra gerçekleştirme şöyle yapılır: Varlık kümeleri veri tabanı tablolarına, öz nitelikler de o tabloların sütunlarına dönüştürülür. İlişkilerin çoğullamasına göre tablolar arasındaki ilişkiler belirlenir. Hangi sütunların anahtar, hangilerinin yabancı anahtar olacağı da böyle ortaya çıkar.

13.1 Bire Bir İlişki

Varlıklardan istediğinizin birinci anahtarını alıp ikinci varlık kümesinde yabancı anahtar olarak belirlersiniz. Hangisinin seçileceği tamamen bağlama kalmıştır. Birini seçip öbür tarafa referans göstermek yeterlidir.

Ek bilgi (derste anlatılmadı): Yabancı anahtar sütunu tek başına ilişkiyi bire bir yapmaz. Aynı değer iki satırda bulunabilir ve ilişki fiilen bire çoka dönüşür. Sütun `UNIQUE` tanımlanırsa ikinci satır reddedilir. Bu davranış sqlite3 ile denendi.

13.2 Bire Çok İlişki

Kitap örneğinde anlatılan yöntemdir: N tarafındaki tabloya, 1 tarafındaki tablonun birinci anahtarı yabancı anahtar olarak eklenir.

Hoca bunu yazılı gösterimle ve tabloları çizerek anlattı. N tane personel bir bölümde çalışır, yani bir bölümde N kişi çalışır, ama bir kişi bir bölümde çalışır.

```
Bolum    (bolumId, adi)                                  -- PK: bolumId
Personel (personelId, adSoyad, adres, maas, bolumId)  -- PK: personelId, FK: bolumId
```

Bölüm tablosunun iki kolonu vardır. Altı çizili bölüm ID birinci anahtardır, diğeri addır. İstediğiniz kadar bölüm kodu ve adı eklenebilir.

bolumId	adi
011	BLM
012	EHM

Personel tablosunun beş kolonu vardır: personel ID, ad, adres, maaş ve bölüm ID. Beşinci kolon ilişkiyi kurar. Hocanın satırında adres ve maaş "bilmem ne" diye geçilmiştir.

personelId	ad	adres	maas	bolumId
345	Göksel	...	...	011

Aynı bölüm ID'sini taşıyan N tane personel olabilir. "345 kodlu personel hangi bölümde çalışıyor?" sorusu adım adım şöyle çözülür:

1. **Verilen:** personel ID 345.
2. **Personel tablosunda arama:** 345 benzersiz olduğu için tek bir satıra varılır.
3. **Okuma:** Bu satırın bölüm ID'si 011'dir.
4. **Bölüm tablosunda arama:** ID'si 011 olan satıra gidilir.
5. **Sonuç:** Bölümün adı BLM'dir. 345 kodlu personel BLM adlı bölümde çalışır.

Ek bilgi (derste anlatılmadı): Hocanın elle izlediği bu iki adımlı arama, SQL'de iki tablonun yabancı anahtar üzerinden birleştirilmesiyle tek sorguda yazılır. Aşağıdaki sorgu hocanın tablolarıyla sqlite3'te denendi ve `BLM` döndürdü. Bölüm kodu metin olarak saklanmalıdır, çünkü sayı olarak saklanırsa 011 değeri 11'e dönüşür.

```sql
SELECT b.adi
FROM Personel p JOIN Bolum b ON p.bolumId = b.bolumId
WHERE p.personelId = 345;
```

13.3 Bu Ayrım Neye Yarar?

Benzersiz anahtarlar değişmez, ama diğer alanlar değişebilir. Hoca bölümün kendi tarihinden örnek verdi: Bölümün adı eskiden Bilgisayar Bilimleri ve Mühendisliği idi, 2000 yılından sonra Bilgisayar Mühendisliği oldu. Bölüm tablosundaki ad güncellendiğinde, o bölüme bağlı kayıtlar sorgulanınca artık yeni ad gelir. Kod aynı kaldığı için veri tutarlılığı sağlanmış olur. Bir kişinin adı değişse de durum aynıdır.

Hocanın vurgusu: Kodun değişmemesi esastır, çünkü benzersiz tanımlayıcı odur. Özetle birinci anahtarlar değişemez.

Birinci anahtarın altı çizilir, yabancı anahtar da ayrıca işaretlenir. Yabancı anahtarın anlamı şudur: Bu değeri al, başka bir varlık kümesindeki, burada bölüm kümesindeki birinci anahtara bağla. Bire çok ilişkiler bu şekilde gerçekleştirilir.

14. Çoka Çok İlişki: Film ve Sinema Örneği

Çoka çok ilişkide ilişkinin kendisi de bir tablo, yani bir varlık kümesi hâline gelir. Yöntem adım adım şöyledir:

1. Varlık kümeleri her zamanki gibi tablolara dönüştürülür.
2. İlişki, yeni bir üçüncü tablo olarak oluşturulur. Bütün nitelikler yine sütun olur.
3. İlişkiye ait tanımlayıcı nitelikler varsa bu tabloya sütun olarak eklenir.
4. Varlıkların birinci anahtarları ilişki tablosuna yabancı anahtar olarak eklenir. Böylece iki tarafa da erişilebilir.
5. Genellikle bu iki anahtar bir araya getirilerek benzersiz tanımlayıcı oluşturulur. Bu yetmiyorsa ya da kullanımı zorlaştırıyorsa ayrı bir kod numarası alanı açılabilir. Genellikle tercih edilen de budur.

Örnekte filmler ve sinema salonları vardır. Bir film pek çok sinema salonunda oynayabilir, bir sinema salonunda pek çok film gösterilebilir.

```
Film   (ID, ad, yapimTarihi, yonetmen)   -- PK: ID
Sinema (ID, ad, adres, telefon)          -- PK: ID
Oynar  (FilmID, SinemaID, tarih, saat)   -- FK: FilmID, SinemaID
```

Film tablosunun dört kolonu vardır, N tane satırla aşağı doğru gider. Sinema tablosunun da dört kolonu vardır. Çoka çok ilişkiyi gerçeklemek için arada bir “film oynar sinema” tablosu yapılır.

İlişki tablosunda film ID, sinema ID, tarih ve saat bulunur. Hoca birinci anahtar için iki seçenek verdi:

- İki yabancı anahtar birlikte benzersiz tanımlayıcı olabilir.
- Yabancı anahtarlar yalnız diğer varlık kümelerine giden anahtarlar olarak kalır ve ayrı bir ID alanı açılır. Genellikle bu, daha hızlı olduğu için tercih edilir, çünkü veri tabanı yönetim sistemleri birinci anahtar alanlarını otomatik artan sayılar olarak kendileri üretebilir.

  Ders sonrası not: Bu tabloda tarih ve saat de tutulduğu için film ID ile sinema ID ikilisi benzersiz olmayabilir. Aynı film aynı sinemada ikinci bir seansta oynarsa aynı ikili tekrar eder. sqlite3 denemesinde ikili birincil anahtar bu ikinci satırı reddetti. Ayrı ID alanı ve film, sinema, tarih, saat dörtlüsünün birlikte anahtar olması ise satırı kabul etti.

Hocanın örnek verileri şöyledir. Filmler 10, 11 ve 12 kodlu, sinemalar AA, BB ve CC kodludur.

FilmID	SinemaID	tarih	saat
10	AA	şu tarih	şu saat
10	BB	şu tarih	şu saat
11	AA	şu tarih	şu saat
11	CC	şu tarih	şu saat
12	AA	şu tarih	şu saat

“10 numaralı film hangi sinemalarda oynuyor?” sorusu adım adım şöyle çözülür:

1. **Verilen:** film ID 10. Filmin adının burada önemi yoktur.
2. **İlişki tablosunda arama:** Film ID’si 10 olan satırlar bulunur.
3. **Okuma:** Bu satırların sinema ID’leri AA ve BB’dir.
4. **Sinema tablosunda arama:** Kod numarası AA olan ve BB olan satırlara gidilir.
5. **Sonuç:** Bu sinemaların adı ve telefonu okunur, raporlanır ya da değiştirilir. Çünkü onlara artık kod numarası üzerinden benzersiz olarak erişilebilir.

Hoca, bu konuların Veri Tabanı Yönetim Sistemleri dersinde çok daha detaylı işleneceğini söyledi. Burada amacı, veri tasarımının nasıl gösterileceğini ve tablolara nasıl hızlıca dönüştürüleceğini öğretecek kadar detay vermektir.

15. Zayıf Varlık

Bazı varlıklar uygulamada kendi başına var olamaz, ancak başka bir varlık varsa var olabilir. Bütün nitelikleri bir araya getirilse bile benzersiz bir tanımlayıcı oluşmaz, diğerleriyle karışabilirler. Bunlara zayıf varlık kümesi denir. Zayıf varlık çift çizgili kutuyla gösterilir.

Hocanın örneği fakültedir. Fakülte bir varlıktır, ama üniversiteleri kapsayan bir sistemin içinde tek başına durması çok mantıklı değildir. Çünkü aynı fakülte başka üniversitelerde de olabilir. Kod numaraları, hatta başka her şeyi aynı olabilir. Türkiye'de teknik üniversiteler hariç neredeyse bütün üniversitelerde mühendislikler mühendislik fakültesi altındadır. Mimarlık fakültesi de böyledir. Bu durumda benzersiz tanımlayıcıyı bulamama sorunu doğar. O hâlde fakülte, bir üniversiteyle birlikte yer alan zayıf bir varlıktır. Bir üniversitede N tane fakülte vardır.

Hoca bunun biraz ileri bir detay olduğunu, ama gerekirse modellemede kullanılabilmesi için atlamadan anlatmak istediğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Yaygın gösterimde zayıf varlığı sahibine bağlayan ilişki büyük olasılıkla çift çizgili eşkenar dörtgenle çizilir. Zayıf varlığın kendi kodu da büyük olasılıkla kısmi anahtar olarak adlandırılır ve kesikli çizgiyle altı çizilir.

Gerçekleştirme çok basittir. Fakültenin numarası ve adı vardır, ama bunlar onu benzersiz tanımlamaya yetmez. Fakülte yalnız üniversiteyle birlikte yaşayabildiğine göre, bağlı olduğu üniversitenin numarası alınıp fakülte tablosuna eklenir. Birinci anahtar, ayrı kolonlar olan üniversite no ve fakülte no'nun ikisidir.

```
Fakulte (universiteNo, fakulteNo, ad)   -- PK: universiteNo + fakulteNo
```

Yani tablo "şu üniversitedeki şu kodlu fakültenin adı nedir?" diye okunur. Yapılan iş, onsuz yapamadığımız varlığın birinci anahtarını alıp kendimize eklemektir. Böylece Yıldız Teknik Üniversitesi'nin şu kodlu fakültesinin adı Elektrik-Elektronik, başka kodlu fakültesinin adı Gemi İnşaatı, İstanbul Teknik Üniversitesi'nin aynı ya da başka kodlu fakültesinin adı Elektrik-Elektronik diyebiliriz. Üniversite koduyla birleşince hepsi benzersiz olur.

16. Veri Tasarımının Özeti

Hoca konuyu şöyle bağladı: Temel olarak varlık-ilişki modeli esas alınır. Hakkında veri tutulması gereken her yapı bir varlık olarak modellenir. Varlıklar doğal olarak ilişkiler içindedir. Bu ilişkiler uygulamada bir bakıma sınıf diyagramlarındaki bağlantılar olarak da gerçekleşir.

Bir varlık-ilişki diyagramında şunlar tanımlanır:

- Varlıklar ve tutulması gereken bütün öz nitelikler.
- Varlıkları benzersiz tanımlayan anahtar öz nitelikler.
- İlişkiler ve çoğullamaları.
- Çoğullu ilişkilerde diğer tarafa erişmeyi sağlayan yabancı anahtar öz nitelikler.

Kutular, çizgiler, elipsler ve eşkenar dörtgenlerle bunlar çizildiğinde veri tasarımı ifade edilmiş olur. Sonra varlık kümeleri veri tabanı tablolarına dönüştürülür. İlişkiler, gerektiğinde yabancı anahtar olan bir kolonla, çoka çok ilişkilerde ise ek tablolarla kurulur. Bu tablolar veri tabanı yönetim sisteminde oluşturulduğunda tasarımın fiziksel hâle getirilmesi tamamlanır.

17. Fizibilite ve Analiz Çalışmasını Belgelemek

Dersin sonunda bir soru üzerine hoca, önceki derslerde görülen fizibilite ve analiz çalışmalarının nasıl gösterilmesi gerektiğini açıkladı. Ana fikri şudur: Sonuç diyagramını koyup geçmek, o çalışmayı yapmış olmak demek değildir.

Fizibilite: Yalnız toplantı raporu, ekip yapısı için bir organizasyon şeması ve bir Gantt diyagramı koymak fizibilite yapılmış demek değildir. Teknik, ekonomik ve zaman fizibilitesi yapılmalıdır. Toplantı raporları da bunların hazırlığının nasıl yapıldığını göstermelidir.

Teknik fizibilitenin altındaki yazılım fizibilitesinde hangi geliştirme ortamlarının neden seçildiği, alternatiflerin neler olduğu, artıları ve eksileri tartışılmalıdır.

Soru: Hoca kendisi öğrenci ağzından sordu: “Biz Java biliyoruz, o yüzden Java seçtik” demek yeterli mi?

Cevap: Hayır. Fizibilite bu demek değildir. Java ile yapılırsa neye mal olur, alternatif olarak .NET teknolojisiyle ya da bir Python ortamında yapılırsa neye mal olur? Eğitim süresi, deneyim ve maliyet işin içine katılarak hepsi birlikte karşılaştırılmalı ve skorlanmalıdır. Fizibilite tablosundaki satırlar ve kutucuklar bu şekilde ortaya çıkar.

Gantt diyagramı: Diyagramın kendisi kadar nasıl oluştuğu da önemlidir. İş paketleri nasıl bulundu? Hangi alt faaliyetler neden var, aralarındaki ilişkiler neler? Onlara hangi kaynaklar atandı, hangi kaynak ne kadar yükleniyor?

Sistem analizi: İş analizi yapılmalı ve belgelenmelidir. Bunun üzerine izlenen yönteme göre şunlar yapılır:

- **İşlevsel çözümleme:** Veri akış diyagramları oluşturulur ve süreç formları yazılır.
- **Nesneye yönelik çözümleme:** Kullanım senaryolarıyla başlanır, kavramsal sınıfların nasıl bulunduğu anlatılır, sonuçta üretilen sınıf diyagramı gösterilir.

  Hocanın vurgusu: Diyagramı koyup geçmek yetmez. Raporda o sonuçlara nasıl ulaşıldığı ve nasıl üretildiği de anlatılmalıdır.

Dersin Toparlaması

Bu derste tasarım aşamasının son konusu olan veri tasarımı tamamlandı. Veri tabanı tasarımının gereksinim analizinden uyarlamaya kadar bütün geliştirme süreciyle paralel yürüdüğü görüldü. Sıra düzensel model, XML ve JSON, nesne modeli ve ORM, NoSQL ve bellek içi sistemler tanıtıldı. Odak ise varlık-ilişki modeli oldu: varlık, öz nitelik, anahtar öz nitelik ve ilişki sembolleri, bire bir, bire çok ve çoka çok ilişkiler, zayıf varlık.

Dönüşüm kuralları dersin en somut kazanımıdır. Varlık kümesi tablo, öz nitelik sütun olur. Bire birde taraflardan birinin birincil anahtarı diğerine, bire çokta 1 tarafının birincil anahtarı N tarafına yabancı anahtar olarak eklenir. Çoka çokta ilişki için ayrı tablo açılır. Zayıf varlıkta sahibin birincil anahtarı eklenir ve kendi koduyla birlikte birincil anahtar olur.

Sıradaki adımda örnek bir proje üzerinden bir vaka çalışması görülecek. Kodlama ve kalite, test aşamaları, yeni sisteme geçiş ve bakım konuları ise ondan sonraki son teorik derste işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.